

Primer Parcial de Programación Orientada a Objetos (72.33)

22/04/2021

Aclaraciones sobre el parcial:

- 1) Comienzo y duración de cada ejercicio
 - **Ejercicio 1:** comienza a las 18:05 hs y tienen 40 minutos para hacerlo
 - **Ejercicio 2:** comienza a las 18:55 hs y tienen 60 minutos para hacerlo
 - **Ejercicio 3:** comienza a las 20:05 hs y tienen 60 minutos para hacerlo
- 2) Los ejercicios que no respeten el enunciado serán considerados inválidos
- 3) Para aprobar el parcial se debe **sumar 5 (cinco) o más puntos**
- 4) Ver todos los ejemplos antes de comenzar a resolver el ejercicio
- 5) Si sienten la necesidad de realizar algún comentario sobre lo que asumieron del enunciado, comentarlo en la respuesta. No se aceptarán aclaraciones por mail o en forma oral
- 6) Si se detecta de acuerdo a la grabación que están usando otra computadora (Respondus no permite el uso de un segundo monitor) el parcial será anulado
- 7) Recuerden no presionar el Alt-tab o Command-Tab
- 8) Cada ejercicio contendrá en la calificación una copia de la respuesta dada y los comentarios sobre la corrección, para que puedan revisarlo
- 9) Las notas y comentarios sobre cada ejercicio los podrán ver directamente desde el Campus. Se fijará día y horario para revisión. En caso de no entender alguna corrección y no poder concurrir en ese horario, pueden enviar mail solicitando revisión, indicando qué ejercicios necesitan revisar, lo mismo si consideran que hay algo mal corregido.

En caso de encontrarse con un error de Respondus enviar un mail (de ser posible adjuntando una foto que capture el error en pantalla) a fmeola@itba.edu.ar indicando el error obtenido.

Ejercicio 1 (3 puntos, 40 minutos)

Se desea modelar un **inventario**, que consiste en un **conjunto de ítems**. Cada ítem se compone de un nombre y un precio. Ya cuenta con la implementación de la clase **Item** (que puede modificarse):

```
public class Item {  
  
    private final String name;  
    private final double price;  
  
    public Item(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
}
```

Al crear un inventario se recibe por parámetro el conjunto de ítems que lo compondrán. El inventario cuenta con métodos para consultar el conjunto de ítems con cierto orden.

Completar los ..., implementar `Inventory` y todo lo necesario para que, con el siguiente programa de prueba:

```
public class InventoryTester {

    public static void main(String[] args) {
        Item item1 = new Item("A", 100.0);
        Item item2 = new Item("B1", 200.0);
        Item item3 = new Item("B2", 200.0);

        Item[] items = new Item[]{item3, item1, item2};

        Inventory inventory = new Inventory(items);

        // Orden natural: ascendente por precio y desempata alfabéticamente por nombre
        Item[] itemsByPriceAsc = inventory.getItems();
        System.out.println(Arrays.toString(itemsByPriceAsc));
        System.out.println("-----");

        // Orden: descendente por precio y desempata alfabéticamente por nombre
        Item[] itemsByPriceDesc = inventory.getItems(.....);
        System.out.println(Arrays.toString(itemsByPriceDesc));
        System.out.println("-----");

        // Orden: alfabético por nombre y desempata ascendente por precio
        Item[] itemsByNameAsc = inventory.getItems(.....);
        System.out.println(Arrays.toString(itemsByNameAsc));
        System.out.println("-----");
    }
}
```

se obtenga la siguiente salida:

```
[{ A >> $100.00 }, { B1 >> $200.00 }, { B2 >> $200.00 }]
-----
[{ B1 >> $200.00 }, { B2 >> $200.00 }, { A >> $100.00 }]
-----
[{ A >> $100.00 }, { B1 >> $200.00 }, { B2 >> $200.00 }]
-----
```

Ejercicio 2 (3.50 puntos, 60 minutos)

Se desea conocer información sobre **programas de TV** en función de su **rating** (obtenido en base a las calificaciones de los usuarios). Ya cuenta con la implementación de la clase **TVShow** (que **no puede modificarse**) que modela a un programa de TV que se compone de un nombre y un rating (que es un valor real entre 0 y 10).

```
public class TVShow {  
  
    private final String name;  
    private final double rating;  
  
    public TVShow(String name, double rating) {  
        this.name = name;  
        if(!validateRating(rating)) {  
            throw new IllegalArgumentException();  
        }  
        this.rating = rating;  
    }  
  
    public static boolean validateRating(double rating) {  
        return rating >= 0 && rating <= 10;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public double getRating() {  
        return rating;  
    }  
  
    @Override  
    public String toString() {  
        return String.format("### %20s : %.2f ###", name, rating);  
    }  
}
```

Se desea modelar un **catálogo de programas de TV** que permita agregar programas al mismo y definir un rango deseado de rating (mínimo y máximo). De esta forma se pueden consultar todos los programas presentes en el catálogo teniendo en cuenta si el rating de cada uno de ellos está o no dentro del rango deseado indicado.

Al crear un catálogo, el rango deseado de rating es entre 0 y 10, es decir, todos los programas de TV del catálogo tendrán un rating dentro de ese rango deseado.

Cuenta con la interfaz **TVCatalog** (que **puede modificarse**)

```
public interface TVCatalog {  
  
    TVCatalog addTVShow(String name, double rating);  
  
    void setRatingRange(double min, double max);  
}
```

Implementar ArrayTVCatalog, TVResponse y todo lo necesario para que, con el siguiente programa de prueba:

```
public class TVCatalogTester {

    public static void main(String[] args) {
        TVCatalog tvCatalog = new ArrayTVCatalog();
        tvCatalog.addTVShow("Mythic Quest", 7.7).addTVShow("Mr. Mayor", 6.1)
            .addTVShow("The Flight Attendant", 7.1);
        System.out.println("-----");

        // Todos los programas de TV dirán In Range
        // porque entran en el rango por defecto.
        for(TVResponse tvResponse : tvCatalog) {
            System.out.println(tvResponse);
        }
        System.out.println("-----");

        // Se setea un rango de rating
        tvCatalog.setRatingRange(7.0, 10.0);
        System.out.println("-----");

        // Los programas de tv con rating entre 7 y 10 dirán In Range.
        // Los demás dirán Out of Range.
        for(TVResponse tvResponse : tvCatalog) {
            System.out.println(tvResponse);
        }
        System.out.println("-----");

        Iterator<TVResponse> secondIt = tvCatalog.iterator();
        System.out.println("-----");

        tvCatalog.addTVShow("Sherlock", 9.1);
        System.out.println("-----");

        // Se setea un rango de rating
        tvCatalog.setRatingRange(8.0, 10.0);
        System.out.println("-----");

        // Los programas de tv con rating entre 7 y 10 dirán In Range.
        // Los demás dirán Out of Range.
        while(secondIt.hasNext()) {
            System.out.println(secondIt.next());
        }
        System.out.println("-----");

        // Los programas de tv con rating entre 8 y 10 dirán In Range.
        // Los demás dirán Out of Range.
        Iterator<TVResponse> thridIt = tvCatalog.iterator();
        while(thridIt.hasNext()) {
            System.out.println(thridIt.next());
        }
        System.out.println("-----");

        TVCatalog emptyTVCatalog = new ArrayTVCatalog();
        System.out.println(emptyTVCatalog.iterator().hasNext()); // Imprime false
        System.out.println("-----");
    }
}
```

```

    try {
        // Lanza un error si los parámetros mínimo o máximo son incorrectos
        tvCatalog.setRatingRange(-1.0, 9.0);
    } catch (Exception ex) {
        System.out.println(ex.getMessage());
    }
    System.out.println("-----");
}
}

```

se obtenga la siguiente salida:

```

-----
###          Mythic Quest : 7.70 ### In Range
###          Mr. Mayor : 6.10 ### In Range
### The Flight Attendant : 7.10 ### In Range
-----
-----
###          Mythic Quest : 7.70 ### In Range
###          Mr. Mayor : 6.10 ### Out of Range
### The Flight Attendant : 7.10 ### In Range
-----
-----
-----
###          Mythic Quest : 7.70 ### In Range
###          Mr. Mayor : 6.10 ### Out of Range
### The Flight Attendant : 7.10 ### In Range
###          Sherlock : 9.10 ### In Range
-----
###          Mythic Quest : 7.70 ### Out of Range
###          Mr. Mayor : 6.10 ### Out of Range
### The Flight Attendant : 7.10 ### Out of Range
###          Sherlock : 9.10 ### In Range
-----
false
-----
Invalid Parameters
-----

```

Ejercicio 3 (3.50 puntos, 60 minutos)

Se desea modelar el **cálculo de expensas** para un edificio. Este sistema debe permitir **crear unidades funcionales (departamentos residenciales o locales comerciales)** para luego poder obtener el importe de las expensas para cada unidad funcional.

Todas las unidades funcionales cuentan con un **nombre** y una cantidad entera positiva de **metros cuadrados** (no es necesario validarlo). Las unidades funcionales **residenciales** cuentan además con una cantidad entera positiva de **ambientes** (no es necesario validarlo).

El cálculo total de las expensas se basa en una serie de reglas:

- Costo por metro cuadrado de la unidad funcional tanto para residenciales como comerciales.
- Multiplicador sobre el total de las expensas exclusivamente para las unidades funcionales comerciales.
- Cargo extra por ambiente sobre el total de las expensas exclusivamente para las unidades funcionales residenciales.

Ya cuenta con la implementación de la clase **Building** (se puede modificar) que contiene estos tres valores necesarios para el cálculo (valores por defecto) y los métodos correspondientes para consultarlos y actualizarlos:

```
public class Building {  
  
    // Costo por metro cuadrado de las U.F. residenciales y comerciales  
    private double m2Expense = 100.0;  
    // Multiplicador sobre total para las U.F. comerciales  
    private double commercialMult = 3.0;  
    // Cargo extra por ambiente sobre total para las U.F. residenciales  
    private double residentialRoomExpense = 200.0;  
  
    public double getM2Expense() {  
        return m2Expense;  
    }  
  
    public void setM2Expense(double m2Expense) {  
        this.m2Expense = m2Expense;  
    }  
  
    public double getCommercialMult() {  
        return commercialMult;  
    }  
  
    public void setCommercialMult(double commercialMult) {  
        this.commercialMult = commercialMult;  
    }  
  
    public double getResidentialRoomExpense() {  
        return residentialRoomExpense;  
    }  
  
    public void setResidentialRoomExpense(double residentialRoomExpense) {  
        this.residentialRoomExpense = residentialRoomExpense;  
    }  
  
}
```

Implementar todo lo necesario para que, con el siguiente programa de prueba:

```
public class ExpensesTester {

    public static void main(String[] args) {
        // Obtengo un nuevo consorcio con los valores por defecto
        // $100 = Costo por metro cuadrado U.F. Residencial y Comercial
        // 3.0 = Multiplicador U.F. Comercial
        // $200 = Cargo extra por ambiente U.F. Residencial
        Building building = new Building();
        System.out.println("-----");

        // Obtengo una U.F. Residencial de nombre "101", de 35 m2 y 2 ambientes
        FunctionalUnit aHouse = building.newResidentialFunctionalUnit("101", 35, 2);
        System.out.println("-----");

        // Expensas para 101 = (35m2 * $100) + (2amb * $200) = 3500 + 400 = $3900
        System.out.println(aHouse);
        System.out.println("-----");

        // $150 = Costo por metro cuadrado U.F. Residencial y Comercial
        building.setM2Expense(150.0);
        System.out.println("-----");

        // Expensas para 101 = (35m2 * $150) + (2amb * $200) = 5250 + 400 = $5650
        System.out.println(aHouse);
        System.out.println("-----");

        // Obtengo una U.F. Comercial de nombre "C70", de 20 m2
        FunctionalUnit aStore = building.newCommercialFunctionalUnit("C70", 20);
        System.out.println("-----");

        // Expensas para C70 = (20m2 * $150) * 3 = 3000 * 3 = $9000
        System.out.println(aStore);
        System.out.println("-----");

        // $400 = Cargo extra por ambiente U.F. Residencial
        building.setResidentialRoomExpense(400.0);
        System.out.println("-----");

        // Expensas para 101 = (35m2 * $150) + (2amb * $400) = 5250 + 800 = $6050
        System.out.println(aHouse);
        System.out.println("-----");

        // Expensas para C70 = (20m2 * $150) * 3 = 3000 * 3 = $9000
        System.out.println(aStore);
        System.out.println("-----");

        // 5.0 = Multiplicador U.F. Comercial
        building.setCommercialMult(5.0);
        System.out.println("-----");

        // Expensas para C70 = (20m2 * $150) * 5 = 3000 * 5 = $15000
        System.out.println(aStore);
    }
}
```

se obtenga la siguiente salida:

```
-----  
-----  
Residencial: U.F. 101 ( 35 m2 ) : $3900.00  
-----  
-----  
Residencial: U.F. 101 ( 35 m2 ) : $5650.00  
-----  
-----  
Comercial: U.F. C70 ( 20 m2 ) : $9000.00  
-----  
-----  
Residencial: U.F. 101 ( 35 m2 ) : $6050.00  
-----  
Comercial: U.F. C70 ( 20 m2 ) : $9000.00  
-----  
-----  
Comercial: U.F. C70 ( 20 m2 ) : $15000.00
```